Hacettepe University

Department of Electrical and Electronics Engineering

ELE432 - Advanced Digital Design

HW2 - Multicycle Processor Controller

Murathan Yılmaz

2210357037

Date: 26.04.2026

Q1) Lab Duration

I spent approximately 5 hours on this homework. The workload included understanding the multicycle datapath, designing the Main FSM transitions in SystemVerilog, and debugging the ALU Decoder logic to ensure full compatibility with the provided testbench and textbook specifications.

Q2) Hierarchical SystemVerilog Code

Below is the hierarchical SystemVerilog description of the multicycle controller and its sub-modules.

controller.sv

```
module controller (input  logic       clk, reset,
                   input  logic [6:0] op,
                   input  logic [2:0] funct3,
                   input  logic       funct7b5,
                   input  logic       zero,
                   output logic [1:0] immsrc,
                   output logic [1:0] alusrca, alusrcb,
                   output logic [1:0] resultsrc,
                   output logic       adrsrc,
                   output logic [2:0] alucontrol,
                   output logic       irwrite, pcwrite,
                   output logic       regwrite, memwrite);

    logic branch, pcupdate;
    logic [1:0] aluop;

    // 1. Main FSM Unit
    mainfsm fsm_unit (
        .clk(clk), .reset(reset), .op(op),
        .Branch(branch), .PCUpdate(pcupdate), .RegWrite(regwrite),
.MemWrite(memwrite),
        .IRWrite(irwrite), .AdrSrc(adrsrc),
        .ResultSrc(resultsrc), .ALUSrcB(alusrcb), .ALUSrcA(alusrca),
.ALUOp(aluop)
    );

    // 2. ALU Decoder Unit
    aludec aludec_unit (
        .opb5(op[5]),
        .funct3(funct3),
        .funct7b5(funct7b5),
        .ALUOp(aluop),
        .ALUControl(alucontrol)
    );

    // 3. Instruction Decoder Unit
    instrdec instrdec_unit (
        .op(op), .ImmSrc(immsrc)
    );

    // 4. PCWrite Logic
```

```
    assign pcwrite = pcupdate | (branch & zero);

endmodule
```

mainfsm.sv

```
module mainfsm(input  logic       clk, reset,
               input  logic [6:0] op,
               output logic       Branch, PCUpdate, RegWrite, MemWrite,
               output logic       IRWrite, AdrSrc,
               output logic [1:0] ResultSrc, ALUSrcB, ALUSrcA, ALUOp);

    typedef enum logic [3:0] {
        FETCH   = 4'd0,  // S0
        DECODE  = 4'd1,  // S1
        MEMADR  = 4'd2,  // S2
        MEMREAD = 4'd3,  // S3
        MEMWB   = 4'd4,  // S4
        MEMWRITE= 4'd5,  // S5
        EXECUTER= 4'd6,  // S6
        ALUWB   = 4'd7,  // S7
        EXECUTEI= 4'd8,  // S8
        JAL     = 4'd9,  // S9
        BEQ     = 4'd10  // S10
    } statetype;

    statetype state, nextstate;

    // State Register
    always_ff @(posedge clk or posedge reset) begin
        if (reset) state <= FETCH;
        else       state <= nextstate;
    end

    // Next State Logic
    always_comb begin
        case (state)
            FETCH:   nextstate = DECODE;
            DECODE: begin
                case(op)
                    7'b0000011: nextstate = MEMADR;   // lw
                    7'b0100011: nextstate = MEMADR;   // sw
                    7'b0110011: nextstate = EXECUTER; // R-type
                    7'b0010011: nextstate = EXECUTEI; // I-type ALU
                    7'b1101111: nextstate = JAL;      // jal
                    7'b1100011: nextstate = BEQ;      // beq
                    default:    nextstate = FETCH;
                endcase
            end
            MEMADR: begin
                if (op == 7'b0000011) nextstate = MEMREAD;
                else                  nextstate = MEMWRITE;
            end
            MEMREAD:  nextstate = MEMWB;
            MEMWB:    nextstate = FETCH;
            MEMWRITE: nextstate = FETCH;
```

```
            EXECUTER: nextstate = ALUWB;
            ALUWB:    nextstate = FETCH;
            EXECUTEI: nextstate = ALUWB;
            JAL:      nextstate = ALUWB;
            BEQ:      nextstate = FETCH;
            default:  nextstate = FETCH;
        endcase
    end

    // Output Logic
    always_comb begin
        // Default values to 0
        {Branch, PCUpdate, RegWrite, MemWrite, IRWrite, AdrSrc} = 6'b000000;
        {ResultSrc, ALUSrcB, ALUSrcA, ALUOp} = 8'b00000000;

        case (state)
            FETCH: begin AdrSrc = 1'b0; IRWrite = 1'b1; ALUSrcA = 2'b00;
ALUSrcB = 2'b10; ALUOp = 2'b00; ResultSrc = 2'b10; PCUpdate = 1'b1; end
            DECODE: begin ALUSrcA = 2'b01; ALUSrcB = 2'b01; ALUOp = 2'b00;
end
            MEMADR: begin ALUSrcA = 2'b10; ALUSrcB = 2'b01; ALUOp = 2'b00;
end
            MEMREAD: begin ResultSrc = 2'b00; AdrSrc = 1'b1; end
            MEMWB: begin ResultSrc = 2'b01; RegWrite = 1'b1; end
            MEMWRITE: begin ResultSrc = 2'b00; AdrSrc = 1'b1; MemWrite =
1'b1; end
            EXECUTER: begin ALUSrcA = 2'b10; ALUSrcB = 2'b00; ALUOp = 2'b10;
end
            ALUWB: begin ResultSrc = 2'b00; RegWrite = 1'b1; end
            EXECUTEI: begin ALUSrcA = 2'b10; ALUSrcB = 2'b01; ALUOp =
2'b10;end
            JAL: begin ALUSrcA = 2'b01; ALUSrcB = 2'b10; ALUOp = 2'b00;
ResultSrc = 2'b00; PCUpdate = 1'b1; end
            BEQ: begin ALUSrcA = 2'b10; ALUSrcB = 2'b00; ALUOp = 2'b01;
ResultSrc = 2'b00; Branch = 1'b1; end
        endcase
    end
endmodule
```

aludec.sv

```
module aludec(input  logic       opb5,
              input  logic [2:0] funct3,
              input  logic       funct7b5,
              input  logic [1:0] ALUOp,
              output logic [2:0] ALUControl);

    logic RtypeSub;
    assign RtypeSub = funct7b5 & opb5;

    always_comb
        case(ALUOp)
            2'b00:   ALUControl = 3'b010; // ADD
            2'b01:   ALUControl = 3'b110; // SUB
            default: case(funct3)
```

```
                        3'b000: if (RtypeSub) ALUControl = 3'b110; // sub
                                else          ALUControl = 3'b010; // add,
addi
                        3'b010: ALUControl = 3'b111; // slt, slti
                        3'b110: ALUControl = 3'b001; // or, ori
                        3'b111: ALUControl = 3'b000; // and, andi
                        default: ALUControl = 3'bxxx;
                    endcase
        endcase
endmodule
```

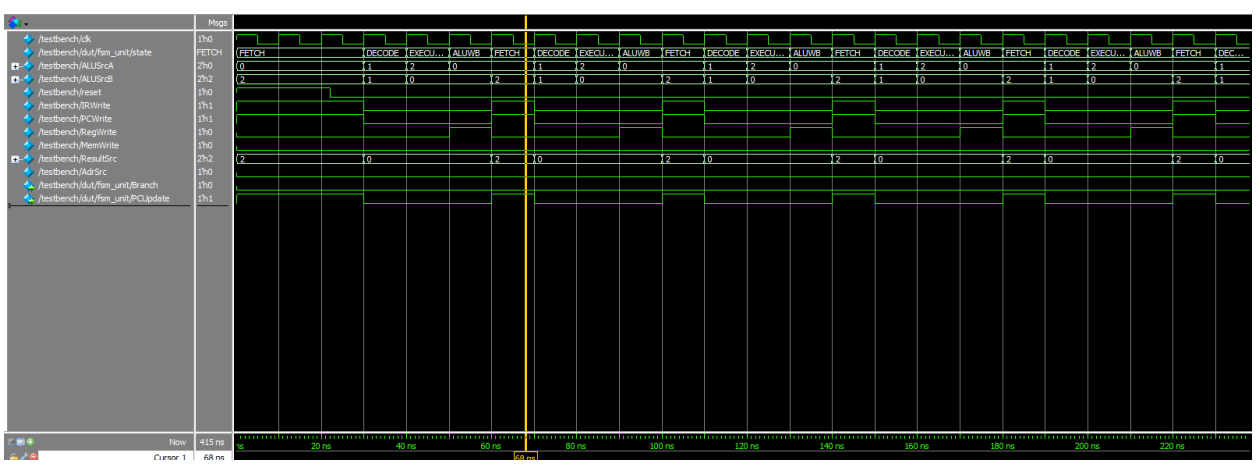
instrdec.sv

```
module instrdec (input  logic [6:0] op,
                 output logic [1:0] ImmSrc);

    always_comb
        case(op)
            7'b0110011: ImmSrc = 2'bxx; // R-type
            7'b0010011: ImmSrc = 2'b00; // I-type ALU
            7'b0000011: ImmSrc = 2'b00; // lw
            7'b0100011: ImmSrc = 2'b01; // sw
            7'b1100011: ImmSrc = 2'b10; // beq
            7'b1101111: ImmSrc = 2'b11; // jal
            default:    ImmSrc = 2'bxx; // ???
        endcase
endmodule
```

Q3) Simulation Results & Test Vectors

The designed multicycle controller successfully passed all provided test vectors. The snapshots below prove that the controller outputs exactly match the expected values for all 40 vectors in controller.tv without any errors.



4. Git Repository Link

All the necessary .sv files and the testbench environment are uploaded to the following Git repository:

`https://github.com/murathany27/ELE432_HW2.git`